Detecção de Bad Smells e Refatoração Segura

Relatório Técnico de Qualidade de Software e Refatoração

Disciplina: Qualidade de Software / Engenharia de Software

Repositório: CleitonSilvaT/bad-smells-js-refactoring

Aluno: Tiago Boaventura Amaral

Matrícula: 847972

1. Análise Manual (Caça aos Bad Smells)

A inspeção visual do arquivo de produção `src/ReportGenerator.js` revelou diversos problemas estruturais clássicos conhecidos como *Bad Smells* (Fowler, 2019). O método monolítico `generateReport` centraliza toda a lógica do módulo em 69 linhas, acumulando responsabilidades divergentes. Abaixo destacam-se os três principais smells:

1.1. Método Longo e Violação de Responsabilidade Única (SRP) BLOATER

Onde ocorre: `ReportGenerator.prototype.generateReport` (linhas 11–68).
Descrição: O método é responsável por orquestrar a montagem do cabeçalho, a iteração de itens, o controle de acesso por perfil (`ADMIN` vs `USER`), a regra de negócio de prioridade, a formatação para CSV e HTML, o somatório financeiro e a geração do rodapé.
Impacto na Manutenção: Qualquer alteração (ex: ajustar a estilização de uma tag HTML ou incluir um novo campo no CSV) exige alterar o mesmo método que contém regras de autorização de negócio, elevando exponencialmente o risco de regressões.
Impacto nos Testes: Torna impossível isolar o teste do cálculo de totais ou da formatação de strings sem instanciar e executar toda a matriz combinatória de usuários e tipos de relatórios.

1.2. Duplicação de Código (Duplicated Code / Violação de DRY) DISPENSABLE

Onde ocorre: Linhas 35–42 e 46–52 de `src/ReportGenerator.js` .
Descrição: A concatenação de colunas CSV (`${item.id},${item.name},${item.value},${user.name}\n`) e o acumulador `total += item.value;` aparecem de forma idêntica dentro do bloco de `ADMIN` e do bloco de `USER` . Além disso, a checagem `reportType === 'CSV'` vs `'HTML'` é repetida 3 vezes ao longo do fluxo.
Impacto na Manutenção: Modificações estruturais no layout do relatório exigem alterações em múltiplos blocos de código. A omissão de um deles gera divergências silenciosas entre o que um perfil de usuário visualiza e o outro.
Impacto nos Testes: Aumenta a redundância dos testes, que precisam validar as mesmas linhas de código duplicadas em múltiplos fluxos de execução.

1.3. Complexidade Ciclomática e Cognitiva Excessiva (Violação do OCP) CHANGE PREVENTER

Onde ocorre: Estrutura de laço e condicionais aninhadas (linhas 27–55).
Descrição: O método encadeia `for` → `if (user.role === 'ADMIN')` → `if (item.value > 1000)` e `if (reportType === 'CSV')` / `else if (reportType === 'HTML')` , acumulando uma árvore de decisão profunda.
Impacto na Manutenção: Fere o *Princípio Aberto/Fechado (OCP)*: para adicionar novos formatos (JSON, XML) ou novos papéis de usuário (GERENTE), é necessário alterar a função monolítica adicionando mais branches aninhados.
Impacto nos Testes: Cada condicional aninhada duplica os caminhos independentes de execução, exigindo uma quantidade desmedida de testes para alcançar cobertura completa de ramos.

2. Relatório da Ferramenta de Detecção (ESLint + SonarJS)

Para fundamentar a análise com métricas de qualidade objetivas, foi configurado o **ESLint** com o plugin oficial do SonarSource, `eslint-plugin-sonarjs`, focado na detecção de anti-padrões e complexidade cognitiva.

```
Terminal — Análise Estática Inicial

$ npx eslint src/

C:\Users\Windows\Documents\faculdade_prog\bad-smells-js-refactoring\src\ReportGenerator.js
  11:3   error  Refactor this function to reduce its Cognitive Complexity from 27 to the 15 allowed  sonarjs/cognitive-complexity
  43:14  error  Merge this if statement with the nested one                                          sonarjs/no-collapsible-if

✖ 2 problems (2 errors, 0 warnings)
```

> **Contribuição do SonarJS:** A métrica de *Complexidade Cognitiva* calcula o esforço mental real para compreender o fluxo do código, atribuindo pesos maiores para quebras de linearidade e aninhamento. O SonarJS revelou que o método `generateReport` atingiu **27 pontos** (limite configurado: 15; ideal para clean code: 5). Além disso, identificou a regra `sonarjs/no-collapsible-if` na linha 43, evidenciando condicionais que poderiam ser fundidas ou eliminadas.

3. Processo de Refatoração Segura

O Bad Smell mais crítico tratado foi a **Complexidade Condicional Excessiva associada à Violação do Princípio Aberto/Fechado (OCP)** e ao **Método Longo**.

3.1. Técnicas Aplicadas

- **Replace Conditional with Polymorphism (Padrão Strategy):** Toda a geração textual de cabeçalho, linhas e rodapé foi encapsulada em classes formatadoras (`CsvReportFormatter` e `HtmlReportFormatter`). Um dicionário mapeador `formatters[reportType]` elimina totalmente as checagens repetitivas de tipo de relatório, permitindo suporte a novos formatos sem alterar a classe orquestradora.
- **Extract Method e Decompose Conditional:** A regra de visibilidade por perfil foi extraída para a função pura `isItemVisibleForUser` ; a marcação de itens prioritários foi isolada em `prepareItem` ; e o cálculo matemático foi extraído para `calculateTotal` .
- **Replace Magic Number with Symbolic Constant:** Os literais `500` e `1000` foram substituídos pelas constantes semânticas `USER_VALUE_LIMIT` e `ADMIN_PRIORITY_THRESHOLD` .

ANTES (SRC/REPORTGENERATOR.JS)

```
// 69 linhas, Complexidade 27
generateReport(reportType, user, items) {
  let report = '';
  let total = 0;
  if (reportType === 'CSV') {
    report += 'ID,NOME,VALOR,USUARIO\n';
  } else if (reportType === 'HTML') {
    report += '...';
  }
  for (const item of items) {
    if (user.role === 'ADMIN') {
      if (item.value > 1000) item.priority = true;
      if (reportType === 'CSV') {
        report += `${item.id},...`;
        total += item.value;
      } else if (reportType === 'HTML') {
        // duplicação de total e tags
      }
    } else if (user.role === 'USER') {
      if (item.value <= 500) {
        // duplicação idêntica de CSV
      }
    }
  }
  // mais if/else de rodapé...
  return report.trim();
}
```

DEPOIS (SRC/REPORTGENERATOR.REFACTORED.JS)

```
// Modular, Strategy Pattern, Complexidade <= 1
class CsvReportFormatter {
  renderHeader() { return 'ID,NOME,VALOR,USUARIO\n'; }
  renderRow(item, user) { return `${item.id},${item.name},${item.value},${user.name}\n`; }
  renderFooter(total) { return `\nTotal,,\n${total},,\n`; }
}

class HtmlReportFormatter {
  renderHeader(user) { return `<html>...<h2>Usuário: ${user.name}</h2>...`; }
  renderRow(item) {
    const style = item.priority ? ' style="font-weight:bold;"' : '';
    return `<tr${style}><td>${item.id}</td><td>${item.name}</td><td>${item.value}</td></tr>\n`;
  }
  renderFooter(total) { return `</table>\n<h3>Total: ${total}</h3>\n</body></html>\n`; }
}

generateReport(reportType, user, items) {
  const formatter = formatters[reportType];
  if (!formatter) return '';

  const visibleItems = items
    .filter(item => isItemVisibleForUser(item, user))
    .map(item => prepareItem(item, user));
  const total = calculateTotal(visibleItems);

  const header = formatter.renderHeader(user);
  const body = visibleItems.map(item => formatter.renderRow(item, user)).join('');
  const footer = formatter.renderFooter(total);
  return `${header}${body}${footer}`.trim();
}
```

```
Terminal — Validação ESLint do Código Refatorado

$ npx eslint src/ReportGenerator.refactored.js

✓ 0 problems (0 errors, 0 warnings) — Aprovado com sucesso!
```

4. Validação Contínua com Testes (Rede de Segurança)

Conforme preconizado na metodologia de *Refatoração Segura*, a suíte de testes original foi preservada e duplicada em `__tests__/ReportGenerator.refactored.test.js`. Nenhuma alteração foi realizada nas asserções de teste; alterou-se apenas a importação para direcionar para o código refatorado.

```
● Terminal — Execução Completa dos Testes (Jest)

$ npm test

PASS __tests__/ReportGenerator.refactored.test.js
PASS tests/ReportGenerator.refactored.test.js
PASS tests/ReportGenerator.test.js

Test Suites: 3 passed, 3 total
Tests:       15 passed, 15 total
Snapshots:   0 total
Time:        4.507 s
Ran all test suites.
```

5. Conclusão

A refatoração segura comprova que é plenamente viável transformar um código legado, com alta complexidade e duplicações, em uma arquitetura limpa, extensível e elegante, sem quebrar nenhuma funcionalidade preexistente.

A existência de uma **suíte de testes automatizada como rede de segurança** foi crucial durante todo o processo: qualquer discrepância na geração de tags HTML ou formatação CSV foi imediatamente detectada pelo Jest em tempo real, permitindo correções ágeis. Aliada a ferramentas de análise estática como o **ESLint e SonarJS**, a equipe obtém métricas matemáticas de manutenibilidade que elevam permanentemente a qualidade do produto de software.